Machine Intelligence

3. Informed Search Methods

Daniel Fišer

AALBORG UNIVERSITY

Fall 2025

Agenda

1. Introduction
2. Heuristic Functions
3. Systematic Search
4. Local Search
5. Conclusion

Informed Search: Basic Idea

- Blind Search: Rigid procedure using the same expansion order no matter which problem it is applied to.

  $\rightsquigarrow$ Blind search has zero knowledge of the problem it is solving.

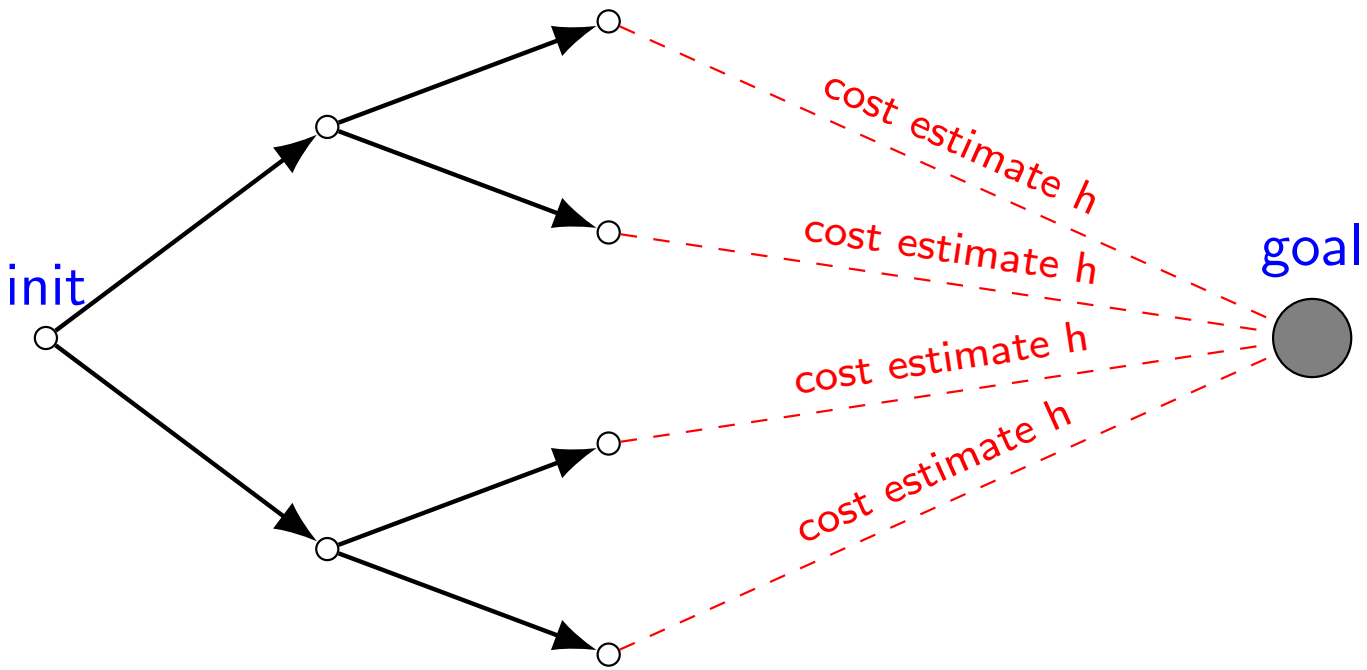
  $\rightsquigarrow$ It can't "focus on roads that go the right direction", because it has no idea what "the right direction" is.

- Informed Search: Knowledge of the "goodness" of expanding a state s is given in the form of a heuristic function $h(s)$, which estimates the cost of an optimal (cheapest) path from s to the goal.

  $\rightsquigarrow$ "$h(s)$ larger than where I came from $\implies$ seems s is not the right direction."

$\rightsquigarrow$ Informed search is a way of giving the computer knowledge about the problem it is solving, thereby stopping it from doing stupid things.

Informed Search: Basic Idea



$\rightsquigarrow$ Heuristic function h estimates the cost of an optimal path from a state s to the goal; search prefers to expand states s with small $h(s)$.

Reminder: Our Agenda for This Topic

- **Previous lecture:** Basic definitions and concepts; blind search.

  $\rightsquigarrow$ Sets up the framework. Blind search is ideal to get our feet wet. It is not wide-spread in practice, but it is among the state of the art in certain applications (e.g., software model checking).

- **This lecture:** Heuristic functions and informed search.

  $\rightsquigarrow$ Search algorithms exploiting the problem-specific knowledge encoded in a heuristic function. Typically much more efficient in practice.

Our Agenda for This Lecture

- **Heuristic Functions:** How are heuristic functions h defined? What are relevant properties of such functions? How can we obtain them in practice?

  $\rightsquigarrow$ Which "problem knowledge" do we wish to give the computer?

- **Systematic Search:** How to use a heuristic function h while still guaranteeing completeness/optimality of the search.

  $\rightsquigarrow$ How to exploit the knowledge in a systematic way?

- **Local Search:** Overview of methods foresaking completeness/optimality, taking decisions based only on the local surroundings.

  $\rightsquigarrow$ How to exploit the knowledge in a greedy way?

Heuristic Functions

Definition (Heuristic Function)

Let Π be a problem with states $\mathcal{S}$. A heuristic function (or just heuristic) for Π is a function $h : \mathcal{S} \mapsto \mathbb{N}_0 \cup \{\infty\}$ such that $h(s_g) = 0$ for every goal state s_g.

The perfect heuristic (or optimal heuristic), denoted as h^*, is the heuristic function assigning every $s \in \mathcal{S}$ the cost of a cheapest path from s to a goal state, or ∞ if no such path exists.

- We also refer to $h^*(s)$ as the goal distance of s.
- $h(s) = 0$ on goal states: If your estimator returns "I think it's still a long way" on a goal state, then its "intelligence" is, um ...
- Return value ∞: To indicate dead ends, from which the goal can't be reached anymore.
- The value of h depends only on the *state* s, not on the *search node* (i.e., the path we took to reach s). I'll sometimes abuse notation writing $h(n)$ instead of $h(n.\text{State})$.

Heuristic Functions: The Eternal Trade-Off

Distance "estimate"? (h is an arbitrary function in principle!)

- We want h to be accurate (aka: informative), i.e., "close to" the actual goal distance.
- We also want it to be fast, i.e., a small overhead for computing h.
- These two wishes are in contradiction!

  $\rightsquigarrow$ Extreme cases? $h = 0$: no overhead at all, completely un-informative. $h = h^*$: perfectly accurate, overhead=solving the problem in the first place.

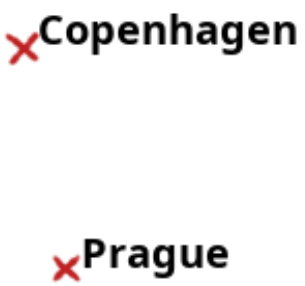
$\rightsquigarrow$ We need to trade off the accuracy of h against the overhead for computing $h(s)$ on every search state s.

So, how to? $\rightsquigarrow$ Given a problem Π, a heuristic function h for Π can be obtained as goal distance within a simplified (relaxed) problem Π'.

How to Relax? Example 1

Problem II: Find a route from Copenhagen to Prague.

How to Relax? Example 1



Relaxed Problem Π': Throw away the map.

How to Relax? Example 1

Heuristic function h: Straight line distance.

How to relax? Example 2

9	2	12	6
5	7	14	13
3	4	1	11
15	10	8	

—

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

- Problem Π: Move tiles to transform left state into right state.
- Relaxed Problem Π': Allow to move each tile to *any cell* in a single move, regardless of the situation.
- Heuristic function h: Number of misplaced tiles. Here: 13.

How to relax? Example 3

9	2	12	6
5	7	14	13
3	4	1	11
15	10	8	

—

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

- Problem Π: Move tiles to transform left state into right state.
- Relaxed Problem Π': Allow to move each tile to any neighbor cell, regardless of the situation.
- Heuristic function h: Manhattan distance. Here: 36.

How to Relax? Example 4

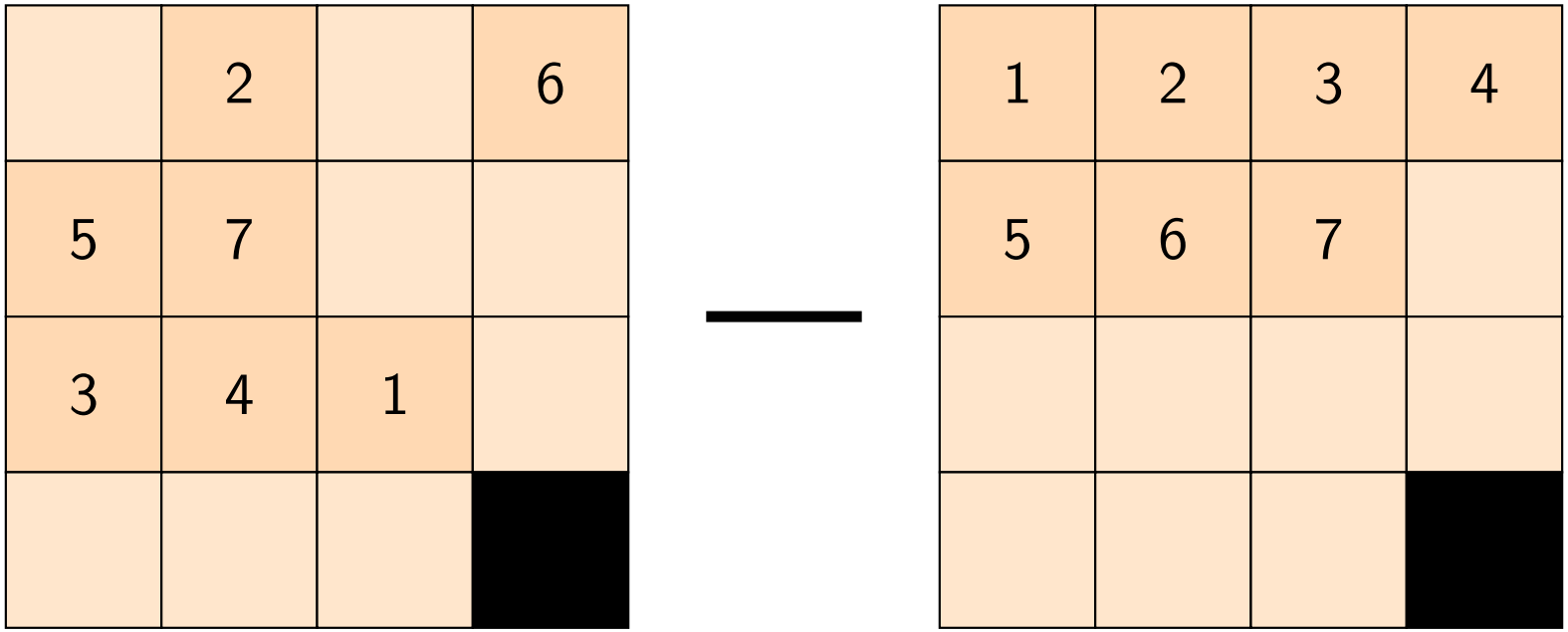
9	2	12	6
5	7	14	13
3	4	1	11
15	10	8	

—

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

Problem Π: Move tiles to transform left state into right state.

How to Relax? Example 4



Relaxed Problem Π': Don't distinguish tiles 8–15.

How to Relax? Example 4

	2		6
5	7		
3	4	1	

—

1	2	3	4
5	6	7	

Heuristic function h: Length of solution to reduced puzzle.

Heuristic Function Pitfalls: Example Path Planning

h^*:

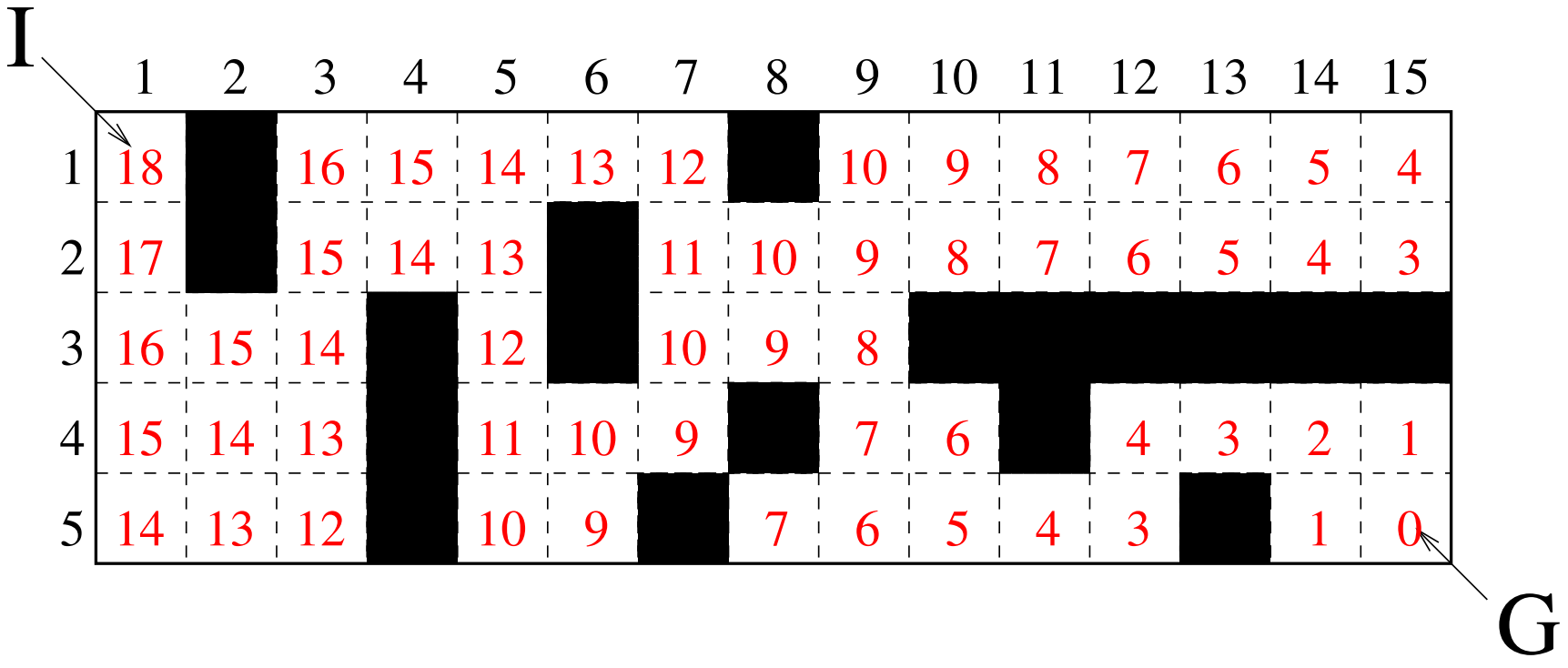
I

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	24		18	17	16	15	14		12	13	14	15	16	17	18
2	23		19	18	17		13	12	11	12	13	14	15	16	17
3	22	21	20		16		12	11	10						
4	23	22	21		15	14	13		9	8		4	3	2	1
5	24	23	22		16	15		9	8	7	6	5		1	0

G

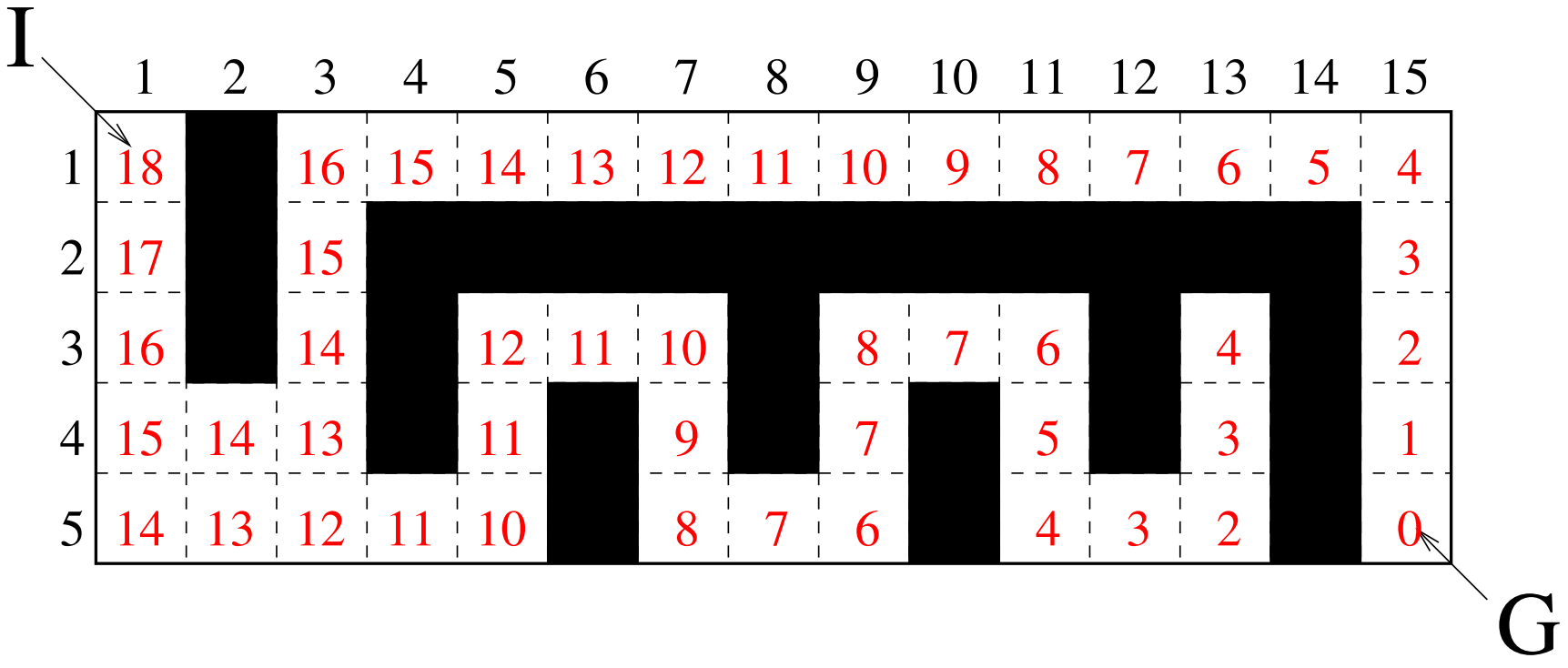
Heuristic Function Pitfalls: Example Path Planning

Manhattan Distance, "accurate h":



Heuristic Function Pitfalls: Example Path Planning

Manhattan Distance, "inaccurate h":



Properties of Heuristic Functions

Definition (safe, admissible, consistent)

Let Π be a problem with state space Θ and states $\mathcal{S}$.
A heuristic h for Π is called:

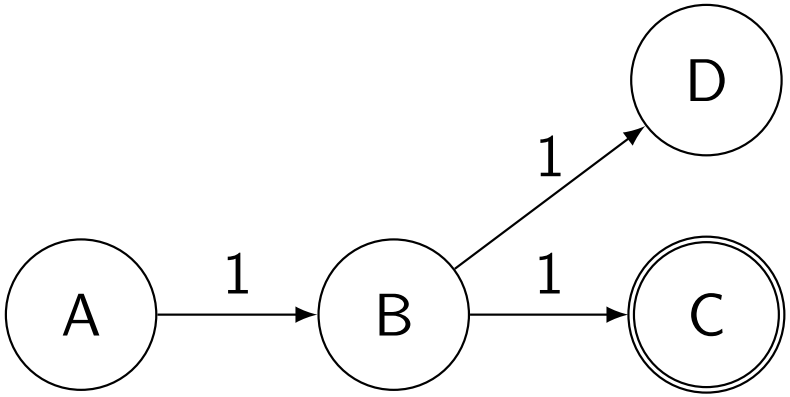
- safe if $h^*(s) = \infty$ for all states $s \in \mathcal{S}$ such that $h(s) = \infty$,
- admissible if $h(s) \leq h^*(s)$ for all states $s \in \mathcal{S}$,
- consistent if $h(s) - h(s') \leq \mathrm{cost}(a)$ for all transitions $s \xrightarrow{a} s'$ in Θ.

$\rightsquigarrow$ Safe heuristics never "lie" about dead-ends. That is, if safe h returns ∞, then we can be sure it is a dead-end.

$\rightsquigarrow$ Admissible heuristics never overestimate the real cost (they are a lower bound on goal distance).

$\rightsquigarrow$ With consistent heuristics, when applying an action a, the heuristic value cannot decrease by more than the cost of a.

Question



	A	B	C	D
h_1	0	1	0	1
h_2	0	1	0	100
h_3	1	2	0	∞
h_4	2	0	0	0
h_5	2	∞	0	0

Question

Which heuristics are safe, consistent, admissible?

- $\rightsquigarrow$ h_1: safe, consistent, admissible
- $\rightsquigarrow$ h_2: safe, consistent, admissible
- $\rightsquigarrow$ h_3: safe, inconsistent ($B \to C$), and inadmissible ($h_3(B) = 2 > 1$)
- $\rightsquigarrow$ h_4: safe, inconsistent ($A \to B$), and admissible
- $\rightsquigarrow$ h_5: not safe, inconsistent, inadmissible

Properties of Heuristic Functions, ctd.

Theorem (consistency implies admissibility)

Let Π be a problem, and let h be a heuristic function for Π.
If h is consistent, then h is admissible.

Proof. We need to show that $h(s) \leq h^*(s)$ for all s. For states s where $h^*(s) = \infty$, this is trivial. For all other states, we show the claim by induction over the length of the cheapest path to a goal state.

Base case: s is a goal state. Then $h(s) = 0$ by definition of heuristic functions, so $h(s) \leq h^*(s) = 0$ as desired.

Step case: Assume the claim holds for all states s' with a cheapest goal path of length n. Say s has a cheapest goal path of length $n+1$, the first transition of which is $s \xrightarrow{a} s'$. By consistency, we have $h(s) - h(s') \leq c(a)$ and thus (a) $h(s) \leq h(s') + c(a)$. By construction, s' has a cheapest goal path of length n and thus, by induction hypothesis, (b) $h(s') \leq h^*(s')$. By construction, (c) $h^*(s) = h^*(s') + c(a)$. Inserting (b) into (a), we get $h(s) \leq h^*(s') + c(a)$. Inserting (c) into the latter, we get $h(s) \leq h^*(s)$ as desired.

Properties of Heuristic Functions: Examples

Admissibility and consistency:

- Is straight line distance admissible/consistent? Yes. Consistency: If you drive 100km, then the straight line distance to Prague can't decrease by more than 100km.
- Is goal distance of the "reduced puzzle" (slide 15) admissible/consistent? Yes. Consistency: Moving a tile can't decrease goal distance in the reduced puzzle by more than 1. Same for misplaced tiles/Manhattan distance.
- Can somebody come up with an admissible but inconsistent heuristic? To-Prague with $h(Copenhagen) = 500$, $h(Aalborg) = 100$.

Inadmissible heuristics:

- Inadmissible heuristics typically arise as approximations of admissible heuristics that are too costly to compute. (We'll see some examples of this in next lecture.)

Exercise Sheet

Do exercises from Exercise Sheet 2: Section 1

Before We Begin

Systematic search vs. local search:

- Systematic search strategies: No limit on the number of search nodes kept in memory at any point in time.

  $\rightsquigarrow$ Guarantee to consider all options at some point, thus complete.

- Local search strategies: Keep only one (or a few) search nodes at a time.

  $\rightsquigarrow$ No systematic exploration of all options, thus incomplete.

Tree search vs. graph search:

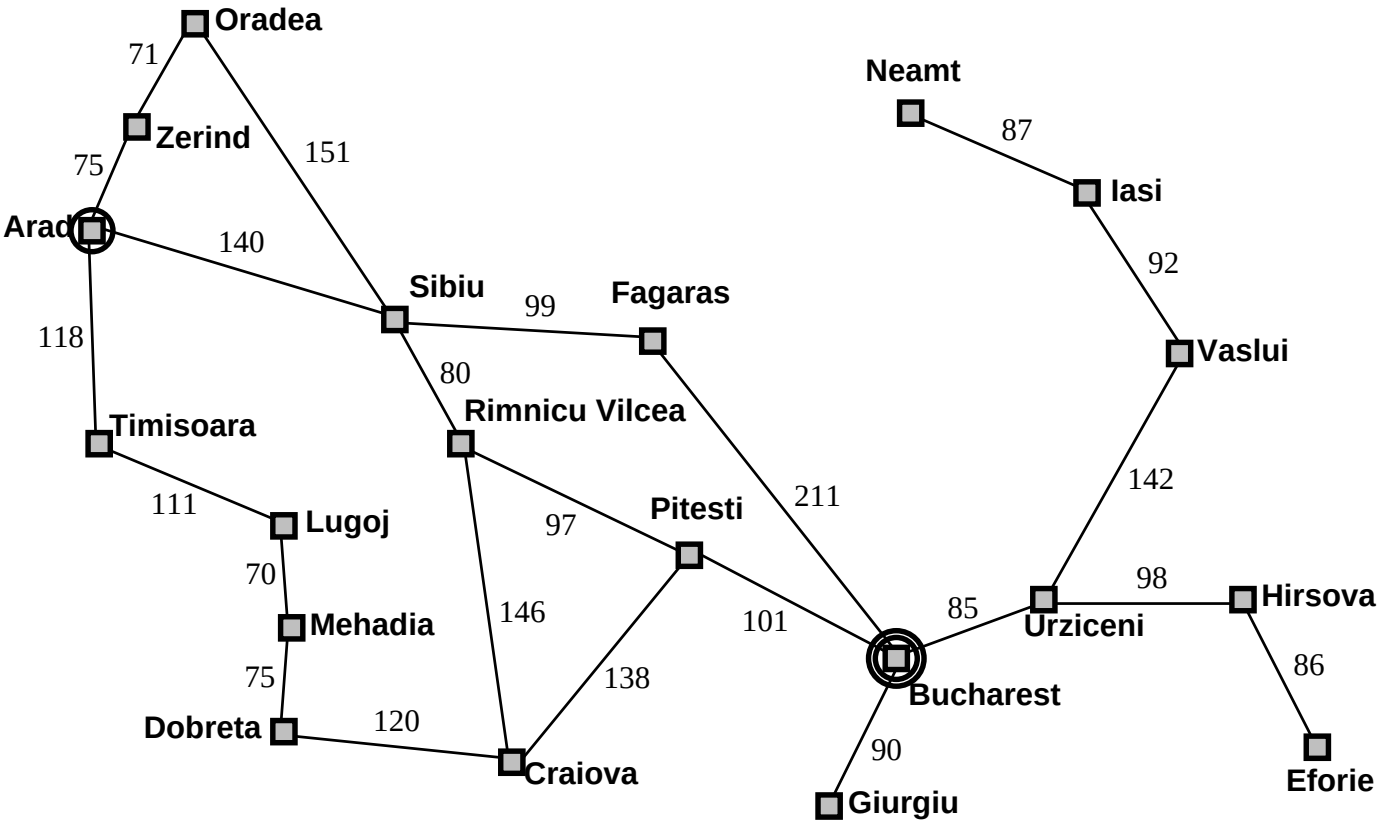
- For the systematic search strategies, we consider graph search algorithms exclusively, i.e., we use duplicate pruning.
- There also are tree search versions of these algorithms. These are easier to understand, but aren't used in practice. (Maintaining a complete open list, the search is memory-intensive anyway.)

Greedy Best-First Search (GBFS)

```
function GreedyBestFirstSearch(heuristic h) returns a solution or "unsolvable"
    n ← InitNode();
    open ← Empty queue of search nodes ordered by ascending h-value;
    /* Let States(open) be a set of all n.State from open                    */
    closed ← ∅ /* Set of already processed states                            */
    Insert n into open;
    while open is not empty do
        n ← Pop(open) /* Pop search node n with lowest g(n)                  */
        if GoalTest(n.State) then return ExtractSolution(n);
        closed ← closed ∪ n.State;
        for each a ∈ Actions(n.State) do
            n′ ← SuccNode(n,a);
            if n′.State ∉ closed and n′.State ∉ States(open) then
                Insert n′ into open /* ordered by h(n′)                      */
    return "unsolvable"
```

- Open list ordered by ascending h.
- GBFS greedily follows the states with lowest h-value.
- $\rightsquigarrow$ Compare to Uniform-cost Search.

Greedy Best-First Search: Route to Bucharest



City	h
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

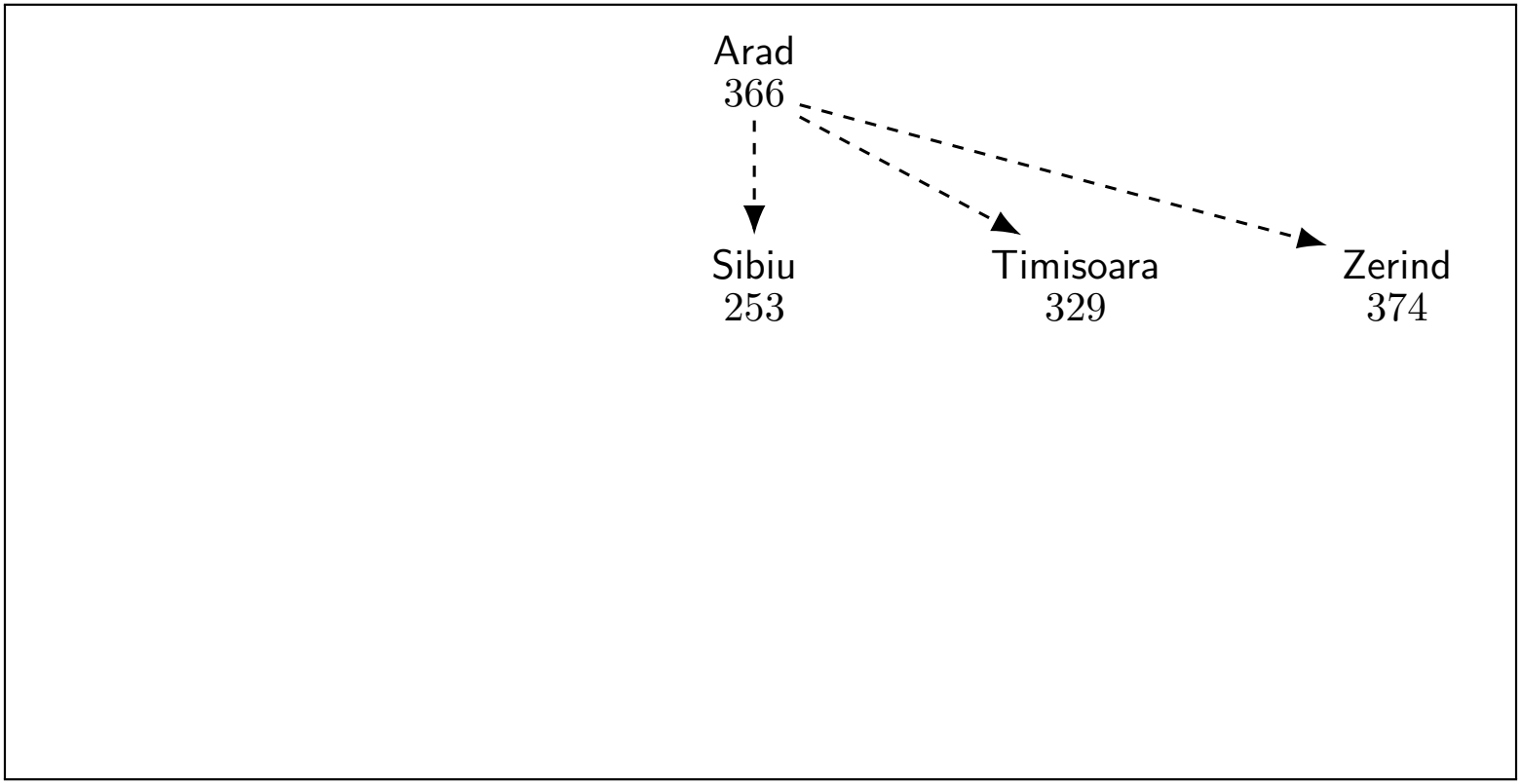
Greedy Best-First Search: Route to Bucharest

Subscripts: h. Red nodes: removed by duplicate pruning.

Arad
366

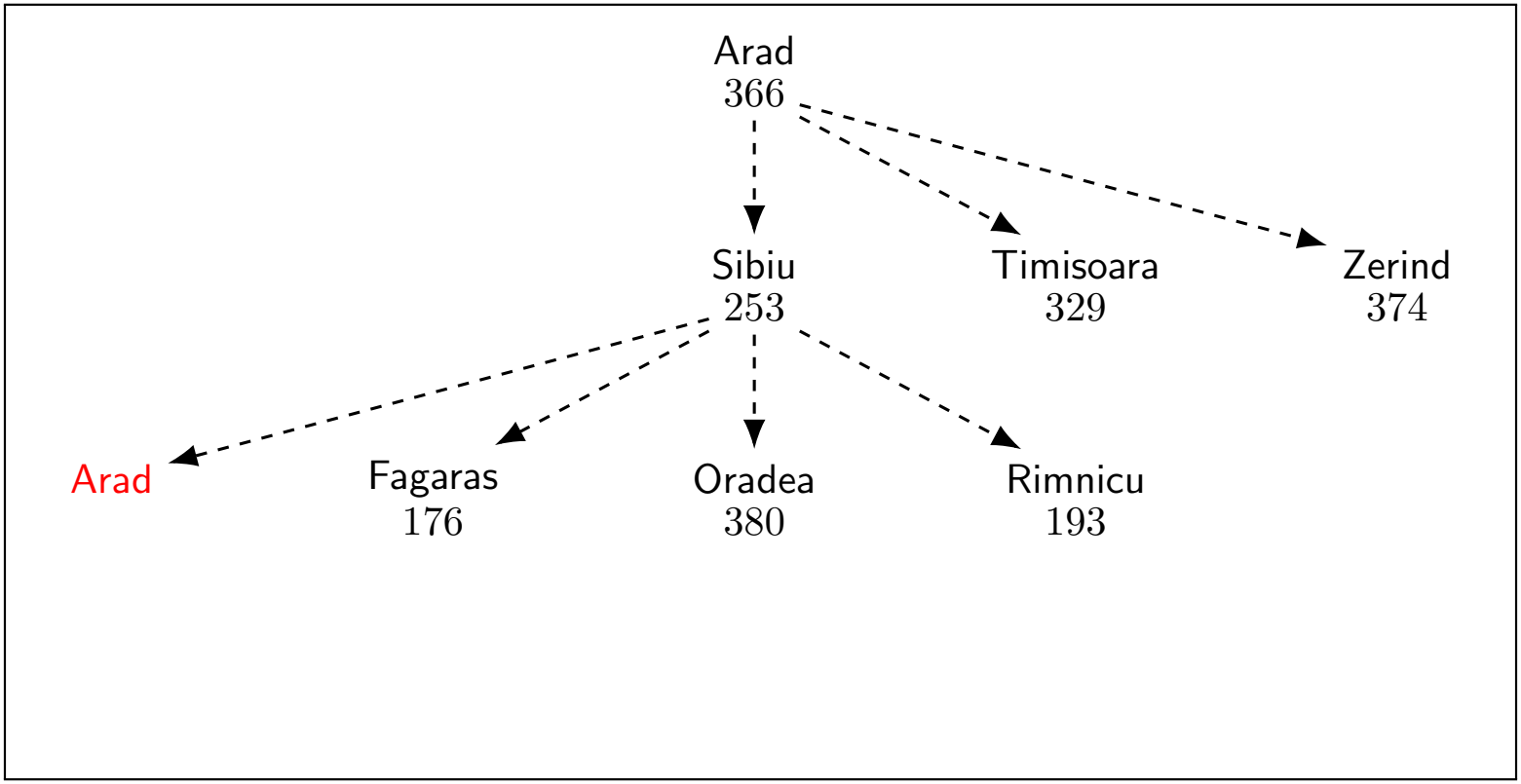
Greedy Best-First Search: Route to Bucharest

Subscripts: h. Red nodes: removed by duplicate pruning.



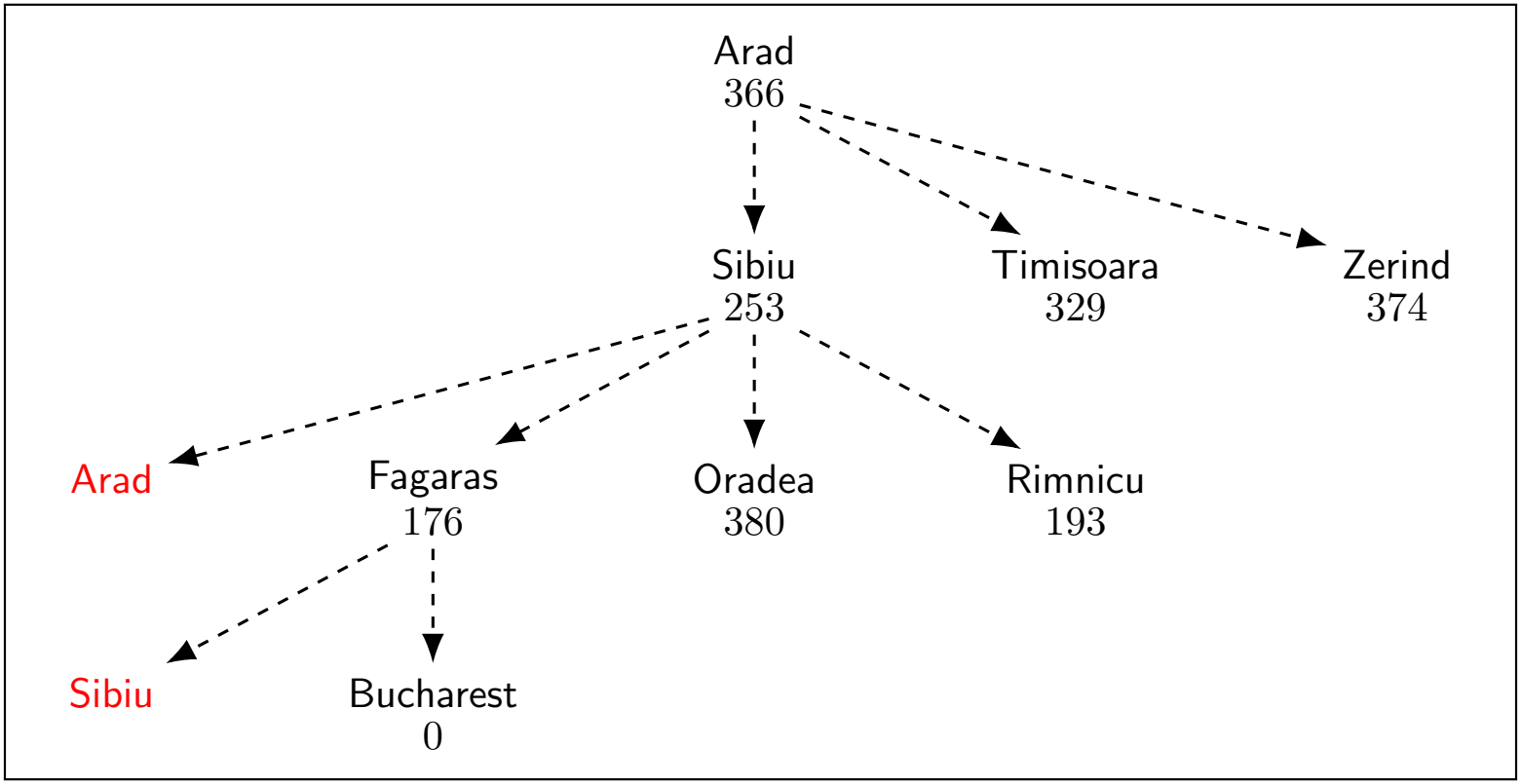
Greedy Best-First Search: Route to Bucharest

Subscripts: h. Red nodes: removed by duplicate pruning.



Greedy Best-First Search: Route to Bucharest

Subscripts: h. Red nodes: removed by duplicate pruning.



Greedy Best-First Search: Guarantees

- Completeness: Yes, thanks to duplicate elimination and our assumption that the state space is finite.
- Optimality? No (h might lead us to Prague via Moscow).

Can we do better than this?

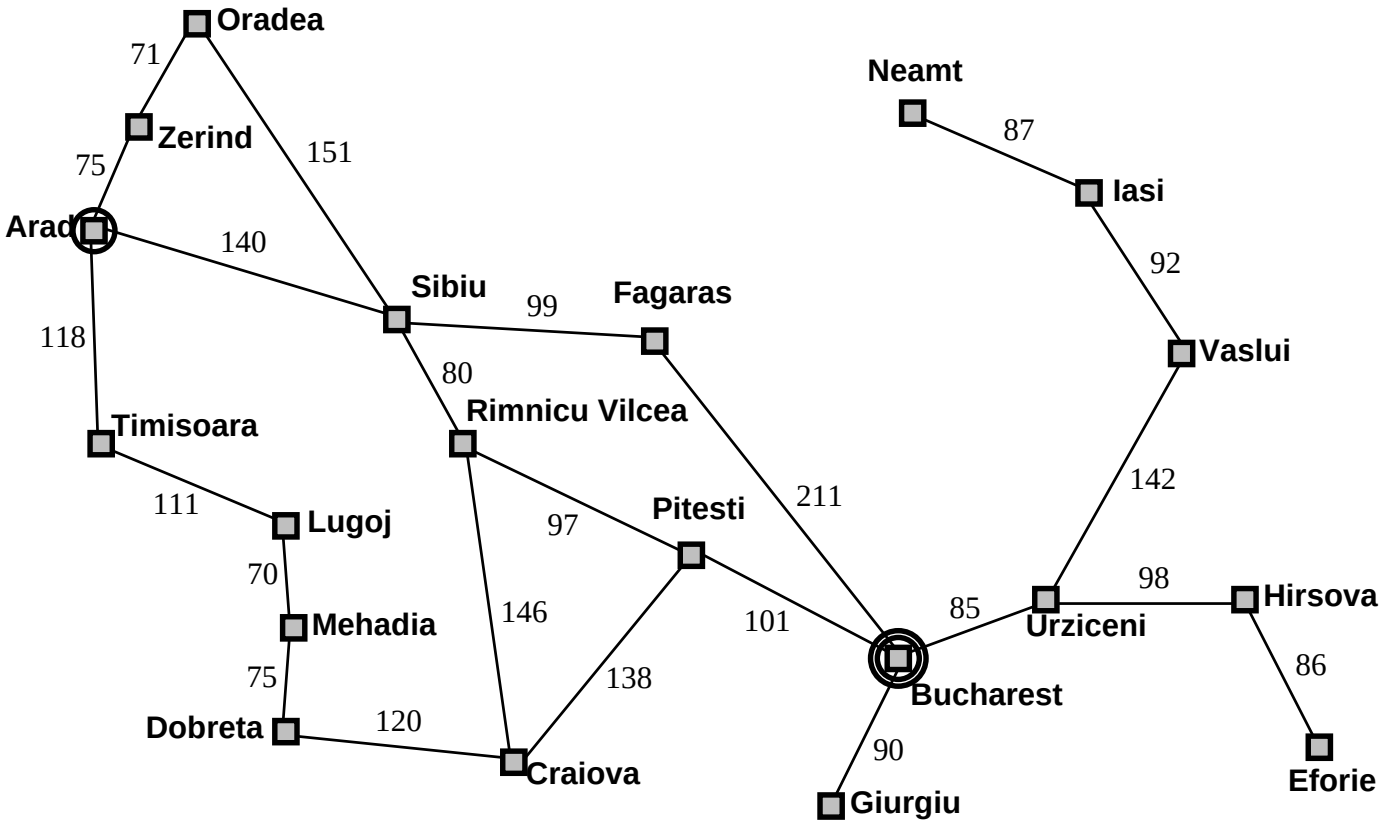
$\rightsquigarrow$ Yes: A* is complete *and* optimal.

A*

```
function A*(admissible heuristic h) returns a solution, or "unsolvable"
    n ← InitNode();
    /* Define:  f(n) = g(n) + h(n.State)                                  */
    open ← Empty queue of search nodes ordered by ascending f-value;
    closed ← ∅ /* Set of already processed states                        */
    Insert n into open;
    while open is not empty do
        n ← Pop(open) /* Pop search node n with lowest g(n)              */
        if GoalTest(n.State) then return ExtractSolution(n);
        closed ← closed ∪ n.State;
        for each a ∈ Actions(n.State) do
            n′ ← SuccNode(n,a) /* Recall that g(n′) = g(n) + cost(a)     */
            if n′.State ∉ closed and n′.State ∉ States(open) then
                Insert n′ into open;
            else if n″ ∈ open s.t. n″.State = n′.State and g(n′) < g(n″) then
                Replace n″ in open with n′;
            else if n″ ∈ closed s.t. n″.State = n′.State and g(n′) < g(n″) then
                /* Re-open n′.State; can trigger only w/ inconsistent h   */
                Remove n″ from closed and insert n′ into open;
    return "unsolvable"
```

- Open list ordered by ascending $g + h$.
- Other than that exactly as Uniform-cost search if h is consistent.

A*: Route to Bucharest



City	h
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

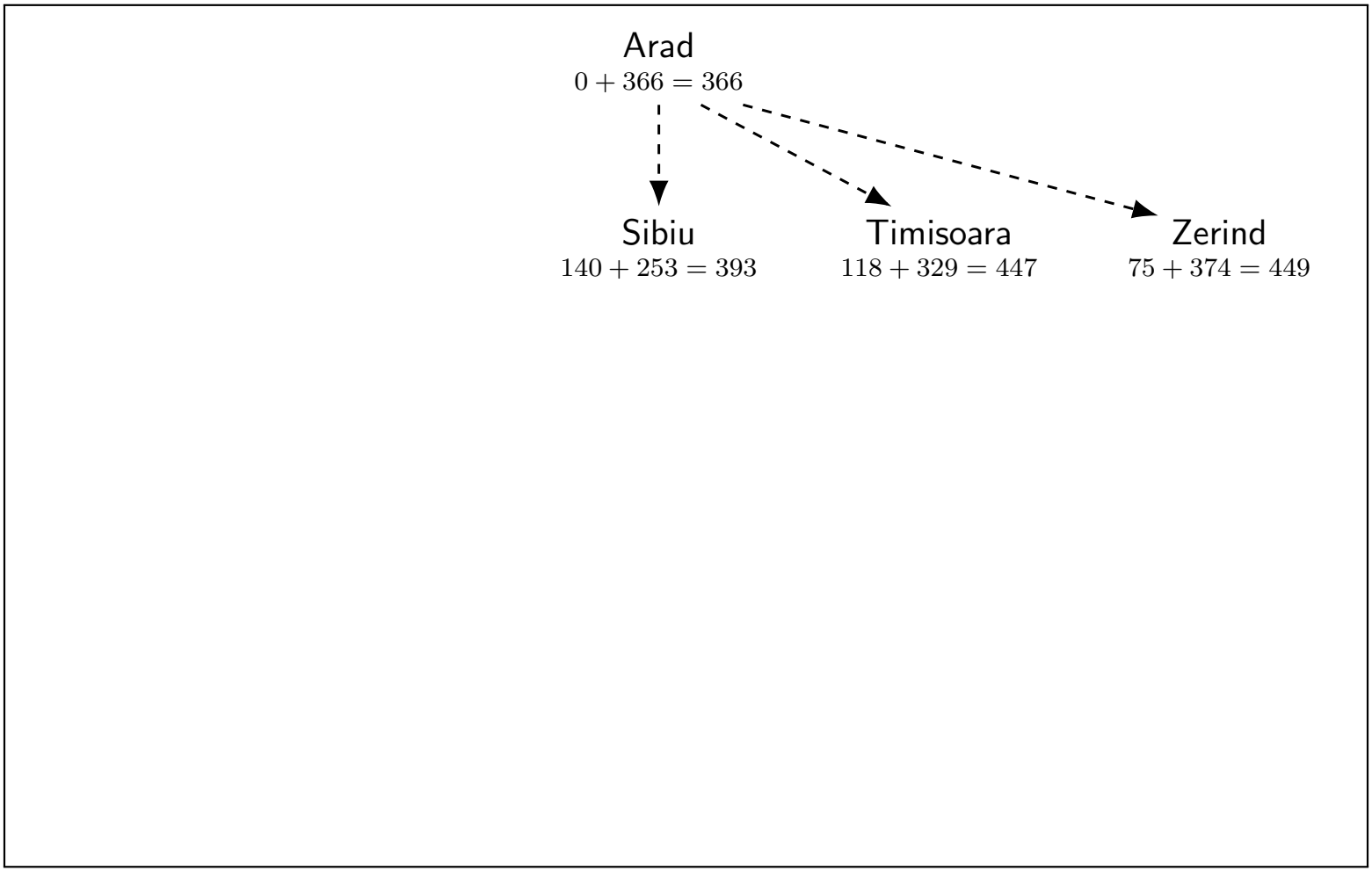
A*: Route to Bucharest

Subscripts: $g + h$. Red nodes: removed by duplicate pruning (without subscript), or because of better path (with subscript g).

Arad
$0 + 366 = 366$

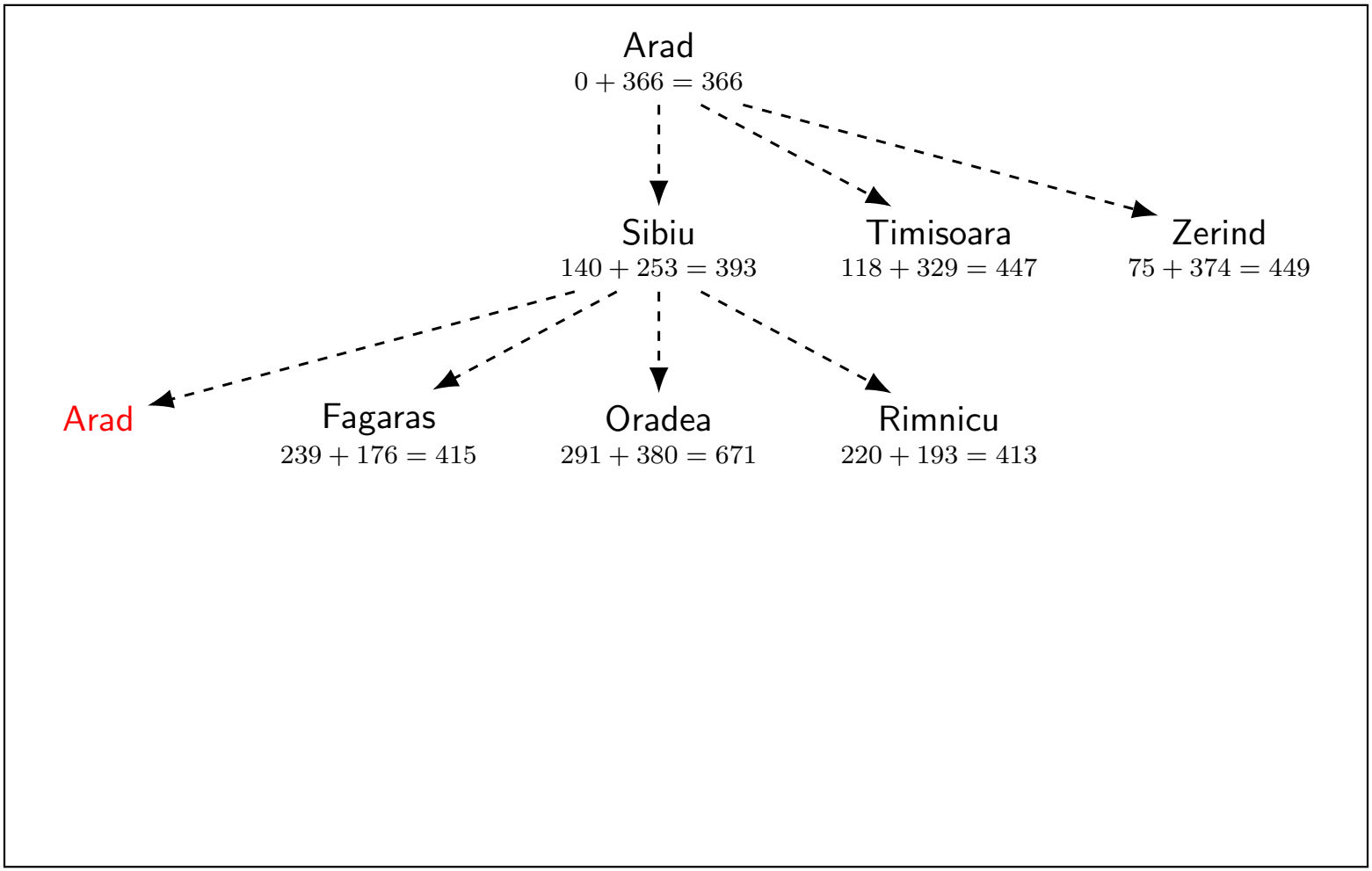
A*: Route to Bucharest

Subscripts: $g + h$. Red nodes: removed by duplicate pruning (without subscript), or because of better path (with subscript g).



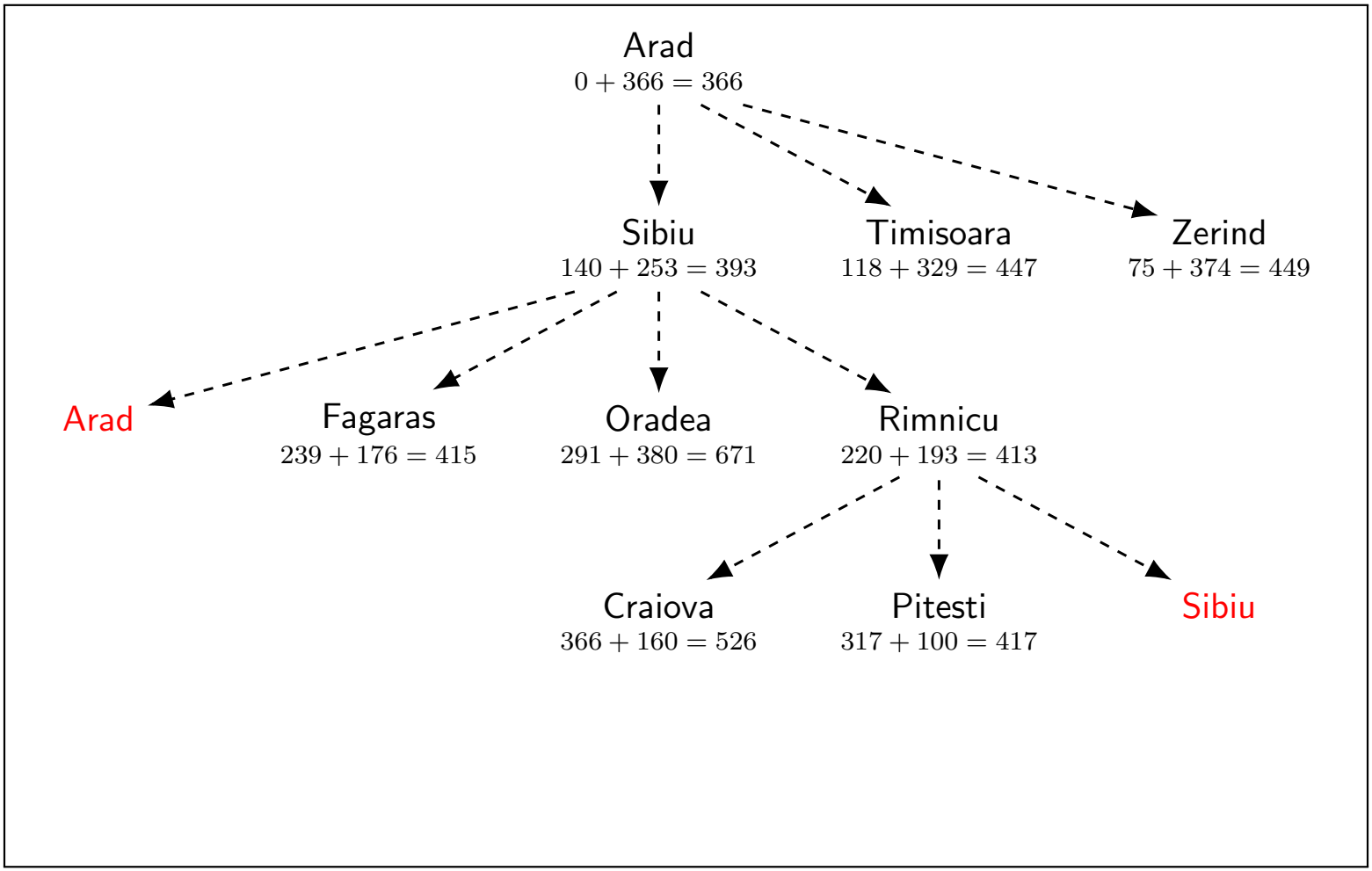
A*: Route to Bucharest

Subscripts: $g + h$. Red nodes: removed by duplicate pruning (without subscript), or because of better path (with subscript g).



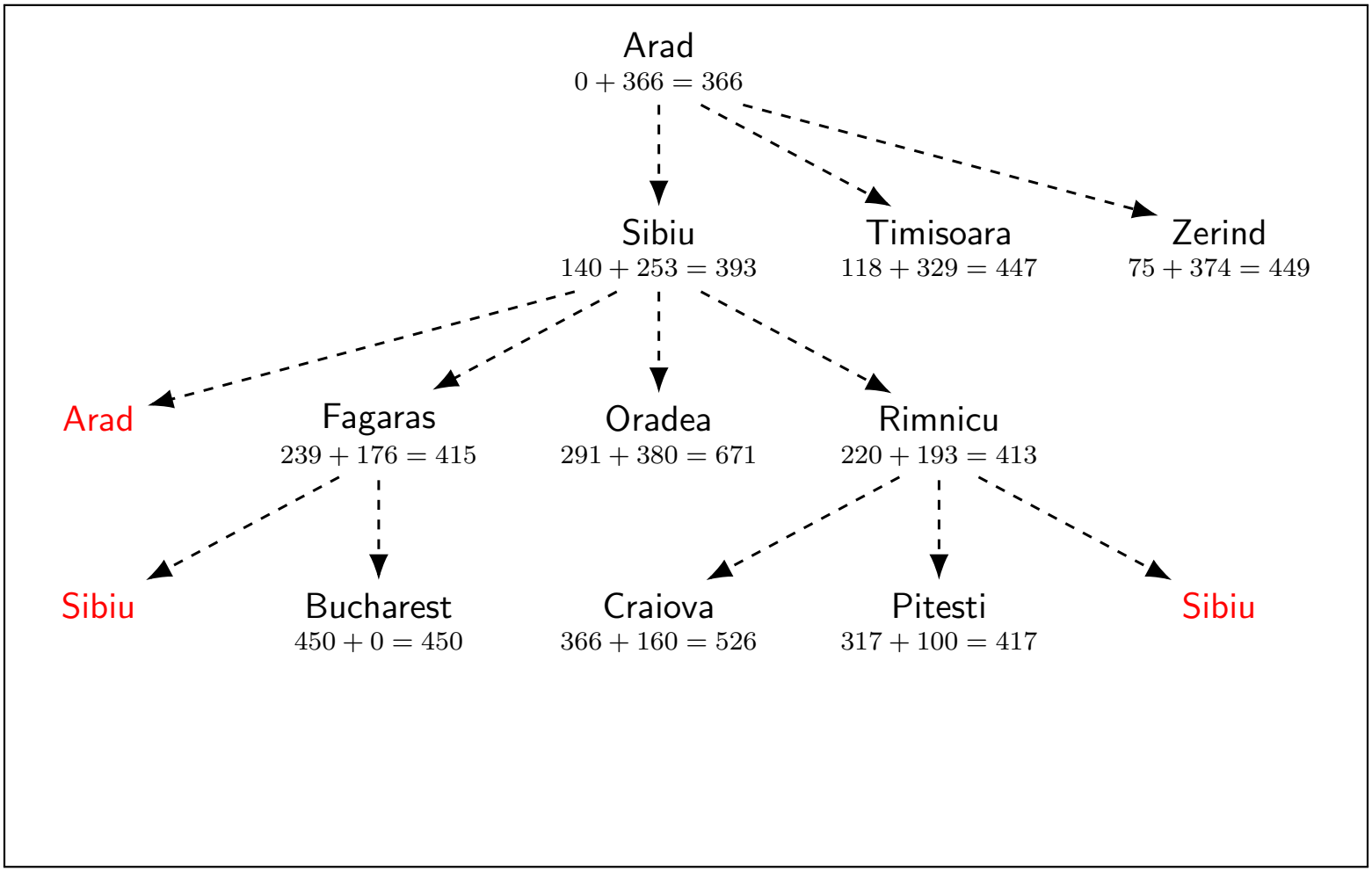
A*: Route to Bucharest

Subscripts: $g + h$. Red nodes: removed by duplicate pruning (without subscript), or because of better path (with subscript g).



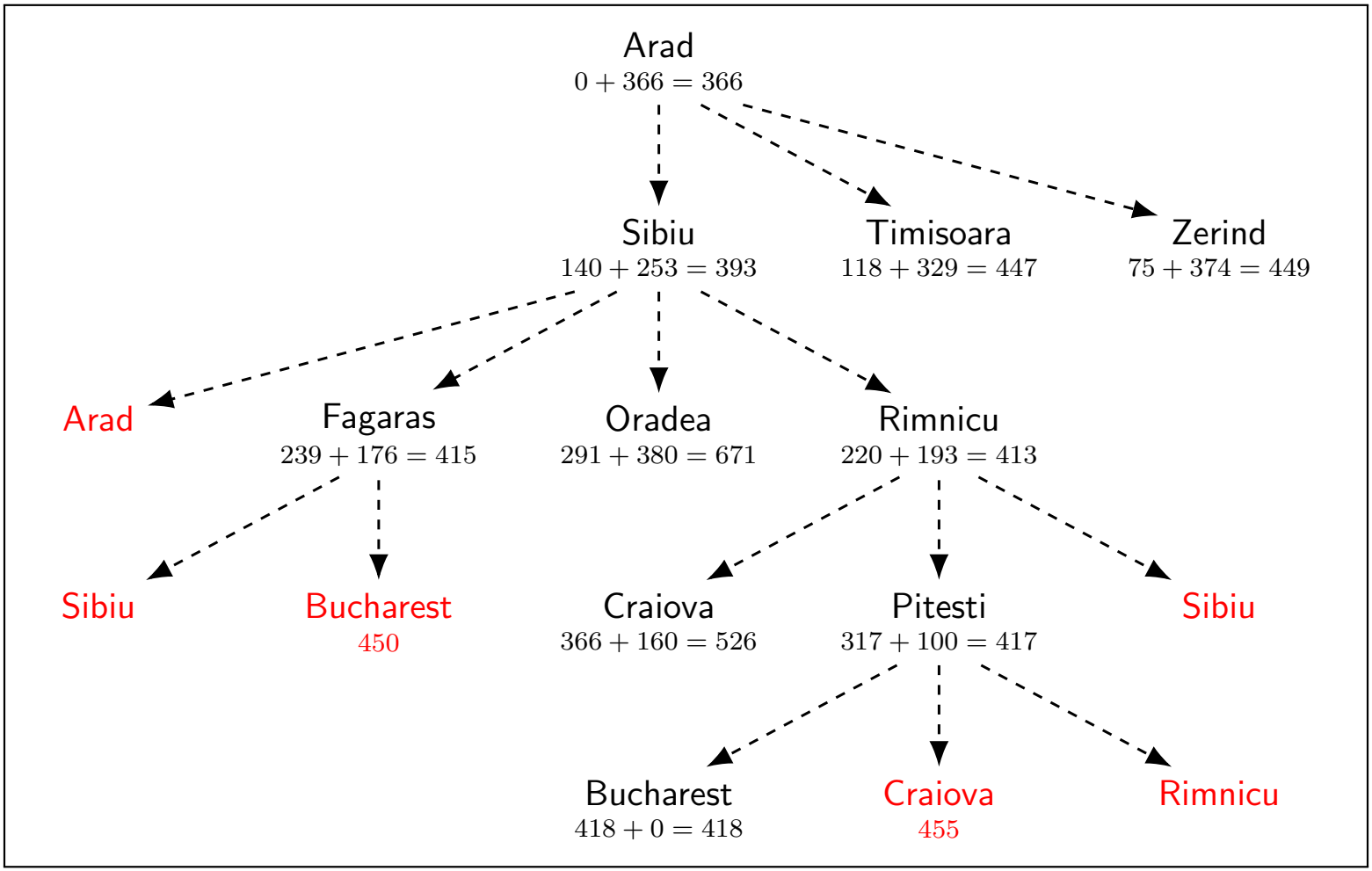
A*: Route to Bucharest

Subscripts: $g + h$. Red nodes: removed by duplicate pruning (without subscript), or because of better path (with subscript g).



A*: Route to Bucharest

Subscripts: $g + h$. Red nodes: removed by duplicate pruning (without subscript), or because of better path (with subscript g).



Questions

Question

If we set $h(s) = 0$ for all states $s \in \mathcal{S}$, what does greedy best-first search become?

(A): Breadth-first search

(B): Depth-first search

(C): Uniform-cost search

(D): A*

$\rightsquigarrow$ h implies no node ordering at all. The search order is determined by how we break ties in the open list. We *basically* get
(A) with FIFO,
(B) with LIFO, and
(C) when ordering on g
(in each case, differences remain in the handling of duplicate states etc).
(D) no, for obvious reasons.

Questions

Question

If we set $h(s) = 0$ for all states $s \in \mathcal{S}$, what does A* become?

(A): Breadth-first search

(B): Depth-first search

(C): Uniform-cost search

(D): Greedy best-first search

$\rightsquigarrow$ (C): Here, the *only* difference between A* and UCS is the use of $g + h$ instead of g when ordering the open list, because $h(s) = 0$ is consistent.

A*: Properties

Completeness: Yes, for same reasons as GBFS or UCS.

Theorem (Optimality of A*)

Let Π be a problem, and let h be a heuristic function for Π. If h is admissible, then the solution returned by A* (if any) is optimal.

Intuitively: $f(n) = g(n) + h(n)$ is a lower bound on the cost of a cheapest solution through the node n. So, when a goal node n_g is expanded (has minimal f-value), we have that $f(n_g) = g(n_g) + h(n_g) = g(n_g) = g^*(n_g)$.

(The difficulty of the proof (which we ommit here) is that (a) $g(n)$ changes as we explore more states, and (b) some states are re-opened in case of inconsistent h.)

A*: Properties

$\rightsquigarrow$ A* is optimal with admissible but inconsistent heuristic *only* if it performs re-openning. If h is consistent, re-openning will never happen.

$\rightsquigarrow$ A* always expands all states with $f(s) = g(s) + h(s) < g^*$ where I is the initial state.

$\rightsquigarrow$ Depending on tie-breaking A* will expand some states with $f(s) = g^*$, but it will not expand any state with $f(s) > g^*$.

$\rightsquigarrow$ With consistent heuristics, A* expands the minimal amount of nodes to find the optimal solution. No other algorithm can expand fewer nodes, unless it has access to other sources of information.

$\rightsquigarrow$ If we use h^* and break ties in favour of smaller h-values, then the search space is linear in the length of the solution.

$\rightsquigarrow$ But if h is "almost perfect", i.e., $|h(s) - h^*(s)| \leq c$ for some constant c, then we may still need to expand exponentially many states.

And now, let's relax a bit . . .

https://qiao.github.io/PathFinding.js/visual/

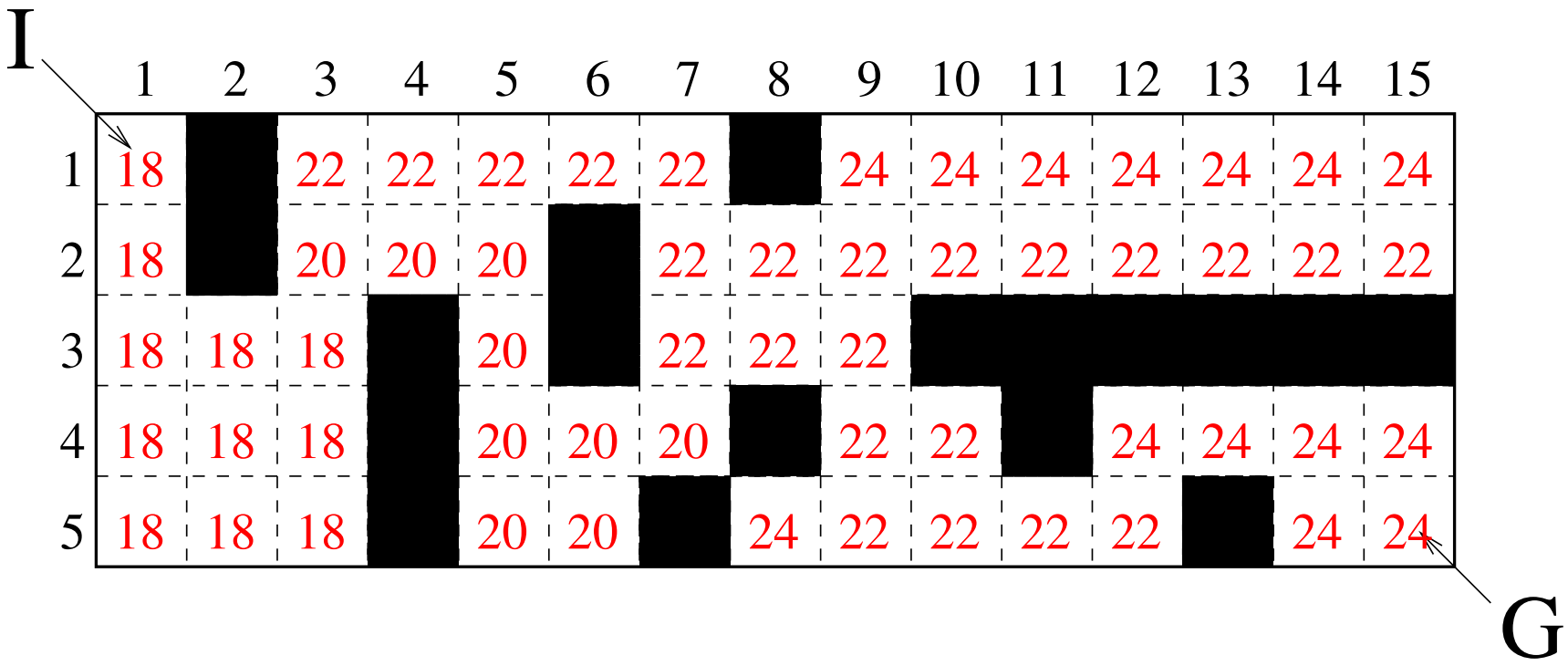
Empirical Performance: A* in the 8-Puzzle

Without Duplicate Elimination; d = length of solution:

	Number of search nodes generated		
	Iterative	A* with	
d	Deepening Search	misplaced tiles h	Manhattan distance h
2	10	6	6
4	112	13	12
6	680	20	18
8	6384	39	25
10	47127	93	39
12	3644035	227	73
14	-	539	113
16	-	1301	211
18	-	3056	363
20	-	7276	676
22	-	18094	1219
24	-	39135	1641

Greedy Best-First vs. A*: Illustration Path Planning

A* ($g + h$), "accurate h":

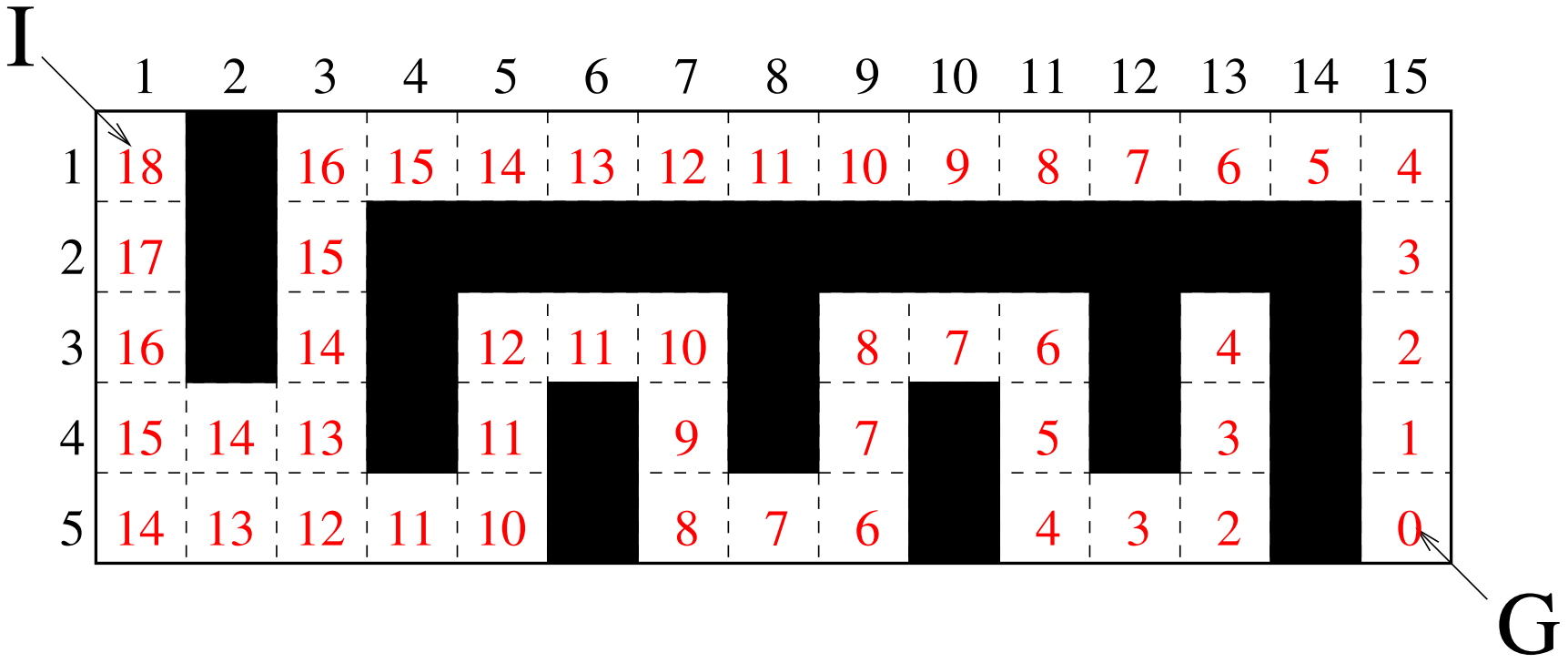


⇝ In A* with a consistent heuristic, $g + h$ always increases monotonically (h cannot *de*crease by more than g *in*creases).

⇝ We need more search, in the "right upper half". This is typical: Greedy best-first search tends to be faster than A*.

Greedy Best-First vs. A*: Illustration Path Planning

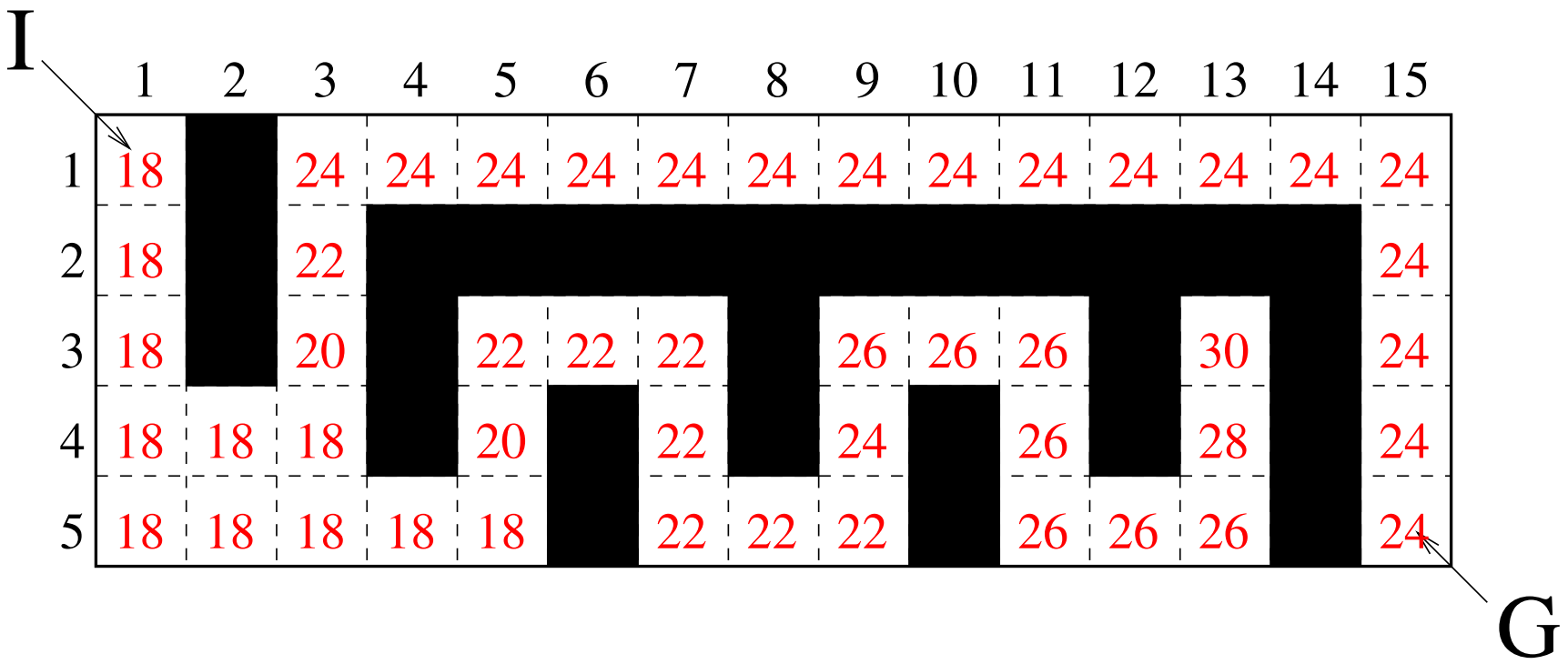
Greedy best-first search, "inaccurate h":



$\rightsquigarrow$ Search will be mis-guided into the "dead-end street".

Greedy Best-First vs. A*: Illustration Path Planning

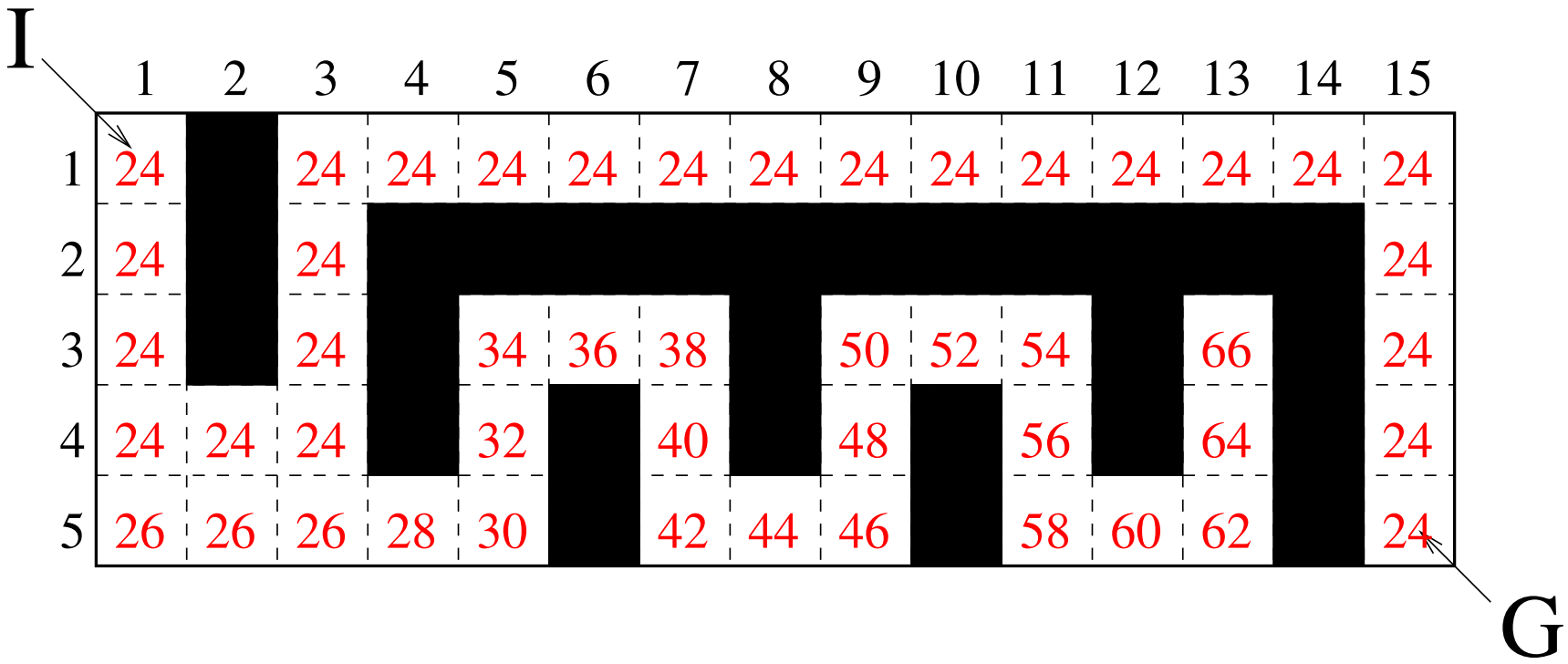
A* **($g + h$), "inaccurate h":**



$\rightsquigarrow$ We will search less of the "dead-end street". For very "bad heuristics", $g + h$ gives better search guidance than h, and A* is faster.

Greedy Best-First vs. A*: Illustration Path Planning

A* $(g + h)$ **using** h^***:**



$\rightsquigarrow$ With $h = h^*$, $g + h$ remains constant on optimal paths.

Questions

Question 1

Is A* always at least as fast as uniform-cost search?

Question 2

Does A* always expand at most as many states as uniform-cost search?

$\rightsquigarrow$ Q1: No, simply because computing h takes time. So the overall runtime may get worse.

$\rightsquigarrow$ Q2: "Yes, but". Considering A* with $h(s) = 0$, both algorithms expand *only* states s where $g^*(s) + h(s) \leq g^*$, and *must* expand all states where $g^*(s) + h(s) < g^*$.

Non-zero h can only reduce the latter. Which s with $g^*(s) + h(s) = g^*$ are explored depends on the tie-breaking used (which state to expand if there is more than one state with minimal $g + h$ in the open list). So the answer is "yes but only if the tie-breaking in both algorithms is the same".

Best-First Search Algorithms: Overview

Algorithm	Uniform-Cost	GBFS	A*	WA*
Order queue by	$g(n)$	$h(n)$	$g(n) + h(n)$	$g(n) + wh(n)$
Complete?	Yes	Yes	Yes[a]	Yes[a]
Optimal?	Yes	No	Yes[b]	No[b,c]

Footnotes:

[a] if h is safe (only returns ∞ for dead-end states)

[b] if h is consistent, or if h is admissible and we re-open nodes when a better path has been found

[c] No, but it guarantees that the solution cost is only sub-optimal by a factor of $w \in \mathbb{R}_0^+$

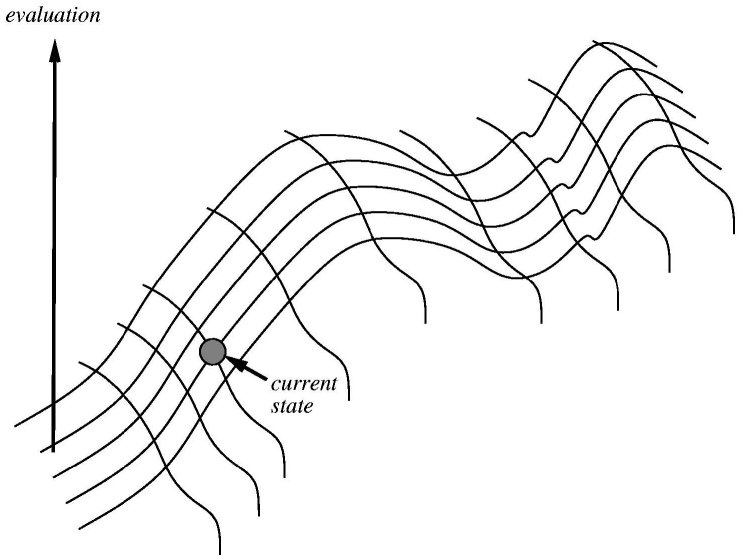
Exercise Sheet

Do exercises from Exercise Sheet 2: Section 2

Local Search

⇝ Sometimes, "going where your nose leads you" works quite well.

What is the computer's "nose"?



⇝ Local search takes decisions based on the h values of immediate neighbor states.

Hill Climbing

```
function HillClimbing(heuristic h) returns a path
    n ← InitNode();
    while true do
        n′ ← among successors n′ of n with minimal h(n′), select one randomly;
        if h(n′) ≥ h(n) then return ExtractSolution(n);
        n ← n′;
```

⇝ Hill-Climbing keeps choosing actions leading to a direct successor state with best heuristic value. It stops when no more immediate improvements can be made.

- Alternative name (more fitting, here): Gradient-Descent.
- Often used in optimization problems where all "states" are feasible solutions, and we can choose the search neighborhood ("child nodes") freely. (Return just n.State, rather than the path to n)

Local Search: Guarantees and Complexity

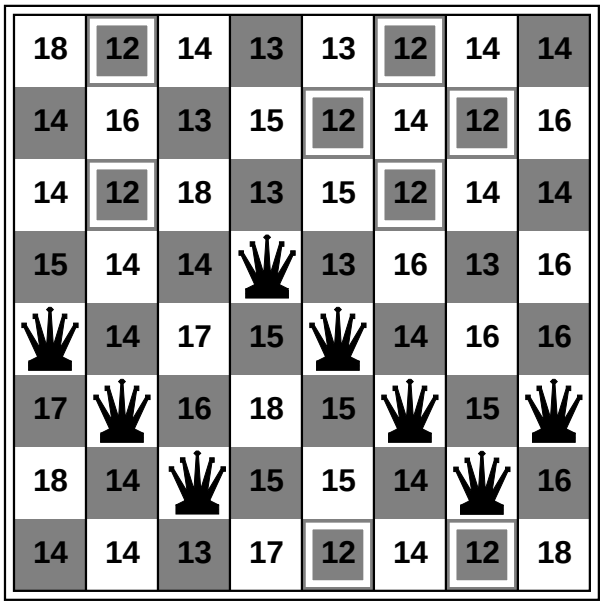
Guarantees:

- Completeness: No. Search ends when no more immediate improvements can be made (= local minimum, up next). This is not guaranteed to be a solution.
- Optimality: No, for the same reason.

Complexity:

- Time: We stop once the value doesn't strictly increase, so the state space size is a bound.
  $\rightsquigarrow$ Note: This bound is (a) huge, and (b) applies to a single run of Hill-Climbing, which typically does not find a solution.
- Memory: Basically no consumption: $O(b)$ states at any moment in time.

Hill Climbing: Example 8-Queens Problem



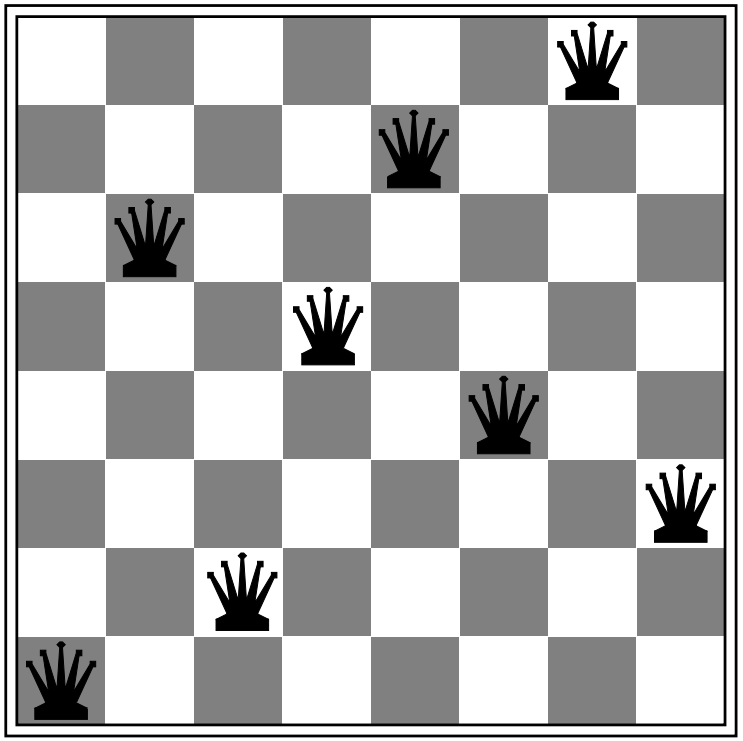
Problem: Place the queens so that they don't attack each other.

Heuristic: Number of pairs attacking each other.

Neighborhood: Move any queen within its column.

$\rightsquigarrow$ Starting from random initialization, solves only 14% of cases.

A Local Minimum in the 8-Queens Problem



$\rightsquigarrow$ Current h value is? 1. To get $h = 0$, we must move either of the 2 queens involved in the conflict. But every such move results in at least 2 new conflicts, so this is a local minimum.

Local Search: Difficulties

Difficulties:

- Local minima: All neighbors look worse (have a worse h value) than the current state (e.g.: previous slide).

  $\rightsquigarrow$ If we stop, the solution may be sub-optimal (or not even feasible). If we don't stop, where to go next?
- Plateaus: All neighbors have the same h value as the current state.

  $\rightsquigarrow$ Moves will be chosen completely at random.

Strategies addressing these:

- Re-start when reaching a local minimum, or when we have spent a certain amount of time without "making progress".
- Do random walks in the hope that these will lead out of the local minimum/plateau.

$\rightsquigarrow$ Configuring these strategies requires lots of algorithm parameters. Selecting good values is a big issue in practice. (Cross your fingers ...)

Hill Climbing With Sideways Moves: Example 8-Queens

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♛	13	16	13	16
♛	14	17	15	♛	14	16	16
17	♛	16	18	15	♛	15	♛
18	14	♛	15	15	14	♛	16
14	14	13	17	12	14	12	18

Heuristic: Number of pairs attacking each other.

Neighborhood: Move any queen within its column.

Sideways Moves: In contrast to slide 42, allow up to 100 consecutive moves in which the h value does not get better.

$\rightsquigarrow$ Starting from random initialization, solves 94% of cases!

$\rightsquigarrow$ Successful local search procedures often combine randomness (exploration) with following the heuristic (exploitation).

Questions

Question

Can local minima occur in route planning with $h :=$straight line distance?

$\rightsquigarrow$ Yes, namely if, from the current position, all roads lead away from the target. For example, following straight line distance from Rome to Egypt will lead you to? A beach in southern Italy . . . (similar for the dead-end street with Manhattan Distance in the path planning "bad case", cf. slide 15).

Exercise Sheet

Do exercises from Exercise Sheet 2: Section 3

Summary

- Search problems require to find a path of actions leading from an initial state to a goal state.
- We considered so-called classical search problems: single-agent, fully-observable, deterministic, static environment. Despite this, they are ubiquitous in practice.
- A problem can be described via its blackbox API, or declaratively, or explicitly. Each method allows to generate the problem's state space.
- For blackbox and declarative descriptions, the state space is exponentially larger than the size of the description, and deciding whether a solution exists is computationally hard (NP and beyond).
- Search algorithms/strategies differ (amongst others) in the order in which they expand search nodes, and in the way they use duplicate elimination. Criteria for evaluating them are completeness, optimality, time complexity, and space complexity.

Summary

- Uniform-cost search is optimal and works like Dijkstra, but building the graph incrementally. Iterative deepening search uses linear space only and is often the preferred blind search algorithm.
- Heuristic functions h map each state to an estimate of its goal distance. This provides the search with knowledge about the problem at hand, thus making it more focussed.
- h is admissible if it lower-bounds goal distance. h is consistent if applying an action cannot reduce its value by more than the action's cost. Consistency implies admissibility. In practice, admissible heuristics are typically consistent.
- Greedy best-first search explores states following lowest h-values. It is complete but not optimal.
- A* explores states by increasing $g + h$. It is complete. If h is consistent, then A* is optimal. If h is admissible but not consistent, then we need to use re-opening to guarantee optimality.

Topics We Didn't Cover Here

- Bounded Sub-optimal Search: Giving a guarantee weaker than "optimal" on the solution, e.g., within a constant factor w of optimal.
- Limited-Memory Heuristic Search: Hybrids of A* with depth-first search (using linear memory), algorithms allowing to make best use of a given amount M of memory, ...
- External Memory Search: Store the open/closed list on the hard drive, group states to minimize the number of drive accesses.
- Real-Time Search: What if there is a fixed deadline by which we must return a solution? (Often: fractions of seconds ...)
- Lifelong Search: When our problem changes, how can we re-use information from previous searches?
- Non-Deterministic Actions: What if there are several possible outcomes?
- Partial Observability: What if parts of the world state are unknown?
- Reinforcement Learning Problems: What if, a priori, the solver does not know anything about the world it is acting in?